

Midterm 2 Report: TRNG on Raspberry Pi

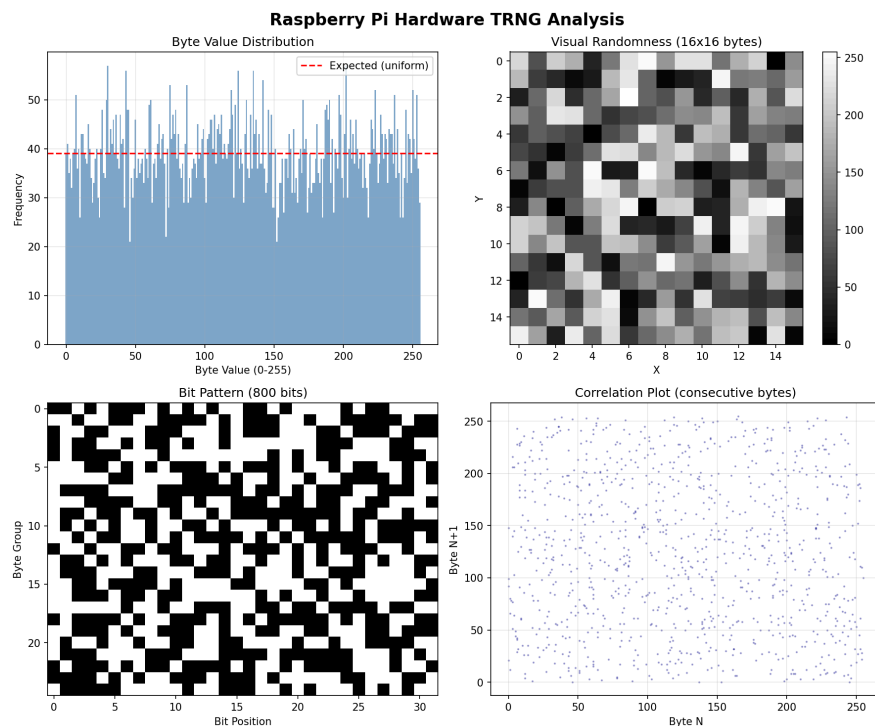
True Random Number Generators (TRNGs) are defined by their reliance on hardware processes to generate a seed, whereas their better known counterpart, pseudorandom number generators, work off known, replicable mathematical algorithms. [1] An issue with replicability is that the same result can be found twice, if not more than that: in cryptography, this is a major security issue, so there is a high level of interest in TRNGs due to their more entropic nature. The four main categories of TRNGs are noise, electrical circuit, quantum, and chaos. [2] There appears to sometimes be category overlap between thermal noise and circuit based TRNG and chaos, due to the nature of entropy from chaos. Chaos can be seen in the natural world and is defined as “the irregular, unpredictable behavior of deterministic, nonlinear systems”, or seemingly “ordered” systems defined by deterministic equations which act in unpredictable ways. [3, pp. 168-169] Simple models can yield unpredictable behavior, and changing initial conditions will completely change the outcome; some examples of chaotic behavior can be seen in thrashing rivers, chemical oscillators, and even a particularly unpredictable day at the New York Stock Exchange could be seen as meeting the mathematical definition of chaos. [3] One example of an unusual use of chaos or entropy as a random number generator is “lavarand,” which was first seen at Silicon Graphics in the late 90s, and used photographing a wall of Lava Lite lava lamps as a basis for seeding computer-based RNG; later, Cloudflare repeated this, more for fun than for true cryptographic security. [4][5] Although photographing lava lamps may be an impractical and potentially insecure way to generate chaotic encryption, it shows the theoretical limits of chaos based TRNG may not exist, since so many random processes exist in nature that we are able to manipulate or track.

FPGAs are a popular technology for developing chaotic random number generators due to their speed and cost in prototyping; an example of a chaos based technology on FPGA is the Boolean chaotic oscillator, which is a system that has nonrepeating chaotic oscillations and is sensitive to initial conditions, like all chaos based functions, but has Boolean-like state transitions and ultimately will output to a Boolean logic function. [1] Thermal noise generators can use the thermal characteristics of diodes or resistors as a source for random voltage and current variations, which can then be used for RNG; however, thermal sources like this, while inexpensive, often rate low in entropy. [2] Other examples of entropy generation for RNG use the scattering of photons and complex optical equipment and photodiodes and radiation detectors to capture “noise” in the form of quantum or light variation. [2] Quantum TRNG is likely the highest in entropy, but requires expensive equipment: examples of this are quantum photon behavior, or “photon arrival time, phase, or polarization” and quantum tunneling, which is electron behavior observed in nanostructures like tunnel diodes. [2] One of the simpler and most famous examples is single photon detection, where a laser passes through a beam splitter and detectors set after the splitter track the paths of the photons; due to quantum uncertainty, there is an inherent randomness in observing the path, and each photon’s path can be mapped directly to a bit. [2]

Physical Unclonable Functions (PUFs) are another interesting element in TRNG. A PUF can be defined as a noisy function embedded into some type of hardware. [6] Although electric circuits tend to be the most practical method of creating PUFs, acoustic, optical, and mechanical PUFs

can exist; the problem is that they are usually not practical to embed in an IC, so cannot be used for device authentication, RNG seeding, etc. [6] Electrically based weak PUFs can be created by “unpredictable” behaviors such as slight variations in manufacturing semiconductors or the gate characteristics of a transistor (ring oscillator PUFs,) or the metastable state of SRAM when it powers on. [7] The benefit of PUFs is in their very low cost for key storage and generation when using circuit-based technologies such as SRAM. [7] PUFs are an example of how TRNGs can be physically embedded into a system for access.

On the Raspberry Pi 5, TRNG uses system-level noise, specifically from thermal noise that exists on the analog components [8]. This relates to one of the examples listed in a previous paragraph when talking about thermal noise generators, which makes sense due to the many components that exist on the Raspberry Pi 5. Below is the implementation of the TRNG on the Raspberry Pi 5:



The graphs above show the Byte Value Distribution, Visual Randomness, Bit Pattern, and the Correlation Plot. In a Byte Value Distribution graph, a flat, or at least relatively flat, graph is expected to show that each value is equally likely to be outputted. From the graph, it is observed that such a phenomenon exists, which means that the TRNG on the Raspberry Pi 5 is quite effective. The other three graphs aim to prove that statement as well. The visual randomness graph, which acts as a heat map, does not show any particular spot as a place where an outcome is likely to occur. This can also be said with the bit pattern, as the outputs are seemingly randomly distributed across the graph. As for the correlation plot, no specific trend exists on the graph, which is what would be observed if the outputs were not random. As such, each of the four graphs proves that TRNG on the Raspberry Pi 5 is an effective way of generating random numbers and introducing chaos into cryptographic algorithms.

Citations:

- [1] F. Yu, L. Li, Q. Tang, Y. Song, and Q. Xu, "A Survey on True Random Number Generators Based on Chaos," *Discrete Dynamics in Nature and Society*, vol. 2019, pp. 1–10, Dec. 2019, doi: <https://doi.org/10.1155/2019/2545123>.
- [2] J. Wu, H. Chai, and Q. Pan, "A Survey of Random Number Generator: Approaches, Tests, Novel Applications in Blockchain and AI Driven Industrial Networks," *Security and Safety*, Nov. 2025, doi: <https://doi.org/10.1051/sands/2025015>.
- [3] R. V. Jensen, "Classical Chaos," *American Scientist*, vol. 75, no. 2, pp. 168–181, 1987, doi: <https://doi.org/10.2307/27854538>.
- [4] I. Peterson, "Lava lamp randomness," *Sci. News*, pp. 92, 1997. Available: <https://www.proquest.com/magazines/lava-lamp-randomness/docview/197491948/se-2>.
- [5] J. Liebow-Feaser, "LavaRand in Production: The Nitty-Gritty Technical Details," *The Cloudflare Blog*, Nov. 06, 2017. <https://blog.cloudflare.com/lavarand-in-production-the-nitty-gritty-technical-details/>
- [6] C. Wachsmann and A.-R. Sadeghi, *Physically Unclonable Functions (PUFs)*. Springer Nature, 2022, pp. 3, 12. doi: <https://doi.org/10.1007/978-3-031-02344-6>.
- [7] C. Herder, M. -D. Yu, F. Koushanfar and S. Devadas, "Physical Unclonable Functions and Applications: A Tutorial," in *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1126-1141, Aug. 2014, doi: <https://10.1109/JPROC.2014.2320516>.
- [8] scruss, "Well — that was unexpected ...: The Raspberry Pi's Hardware Random Number Generator," *We Saw a Chicken ...* (blog), Jun. 7, 2013. https://scruss.com/blog/2013/06/07/well-that-was-unexpected-the-raspberry-pis-hardware-random-number-generator/?utm_source=chatgpt.com.